

Program 1: The GPU Memory (HBM) Calculator

OVERVIEW

Figures out exactly how much GPU memory (VRAM) one copy of the model will consume when turned on.

- **Purpose:** To calculate the total GPU High Bandwidth Memory (HBM) required to run a single instance of a DeepSeek model (such as V4 Flash, V4 Pro, or R1). It is not meant to measure how fast the model will run, just how much space it takes up.
- **Function:** It breaks down the memory footprint into specific categories to ensure it will fit on a graphics card.
- **Usage:** Ask it for a high-level summary (--view summary), a detailed breakdown of where the memory goes (--view components), or how it fits onto specific hardware like the NVIDIA H20 or Huawei Ascend chips (--view hardware).

USAGE EXAMPLES

This tool uses --view to change what data is displayed and allows you to simulate specific memory constraints.

1. The Component Breakdown

To see exactly how much memory goes toward weights, KV cache, and workspace for all default models, use the components view:

```
python3 deepseek_gpu_hbm_calculator.py --view components
```

- **What it does:** Displays a table breaking down the memory footprint (in decimal GB) for each model's disk size, resident weights, quantization metadata, and more.

2. Testing a Specific Model on Specific Hardware

If you only care about running the DeepSeek R1 671B model on a Huawei Ascend 910B 64GB chip, and you want to see if the aggregate hardware can handle a longer context window:

```
python3 deepseek_gpu_hbm_calculator.py --models r1-671b --hardware 910b-64 --average-resident-tokens 8192 --view hardware
```

- **What it does:** Filters the output to only evaluate the r1-671b model on 910b-64 hardware. It also doubles the default average resident tokens to 8192 to see how a larger conversation history impacts the capacity lower bound.

3. Simulating FP8 Materialization (JSON Output)

Simulates a scenario where the V4 Flash model expands its packed FP4 experts into materialized FP8 (1 byte per element) and output the results for another script to read:

```
python3 deepseek_gpu_hbm_calculator.py --models flash-0731 --fp4-resident-bytes-per-element 1.0 --format json
```

- **What it does:** Changes the simulated persistent FP4 expert width to 1.0 (simulated materialized FP8) and outputs the entire calculation as a raw JSON dictionary rather than a formatted table.

SOURCE CODE

```
#!/usr/bin/env python3
"""Per-model GPU HBM sizing for three DeepSeek checkpoints.

The objective is an aggregate accelerator-memory scenario, in decimal GB, for
one model instance. It is not a per-rank topology proof. User demand and replica
throughput are intentionally excluded.

Scenario HBM = resident model tensors
               + quantization/packing metadata
               + runtime/model workspace
               + KV cache for a declared resident-token envelope
               + graph/communication buffers
               + fragmentation/safety reserve

Defaults use official checkpoint metadata and vLLM cache layouts. Runtime,
graph/communication, reserve, and any requested FP4 expansion are simulations
because they cannot be known before profiling the exact image, checkpoint
revision, topology, and hardware. Override them before production approval.

Resident memory is not inferred from native arithmetic support. vLLM's official
recipes validate the packed mixed FP4/FP8 checkpoints on H200 (Hopper), so the
default retains packed FP4 experts in HBM. ``--fp4-resident-bytes-per-element``
can simulate a runtime that materializes FP8 (1 byte) or BF16 (2 bytes) experts.

The default workspace model is:

workspace bytes = model-specific back-fitted bytes/batched-token
```

```
* max_num_batched_tokens
```

The coefficients reproduce the former 3%-of-checkpoint defaults at 16,384 batched tokens. They are SIMULATED continuity values, not measurements. The legacy weight-fraction model remains available with ``--workspace-mode weights``.

The legacy ``per\_tp8\_rank\_average\_gb`` field remains deprecated and always means scenario HBM divided by eight. Allocation averages are informational only; they can hide the maximum-rank residency that determines whether a topology runs.

Quick start:

```
python3 deepseek_gpu_hbm_calculator.py
python3 deepseek_gpu_hbm_calculator.py --view components
python3 deepseek_gpu_hbm_calculator.py --view hardware
python3 deepseek_gpu_hbm_calculator.py --format json
python3 deepseek_gpu_hbm_calculator.py --help
```

Primary evidence:

DeepSeek checkpoint configs and safetensors metadata:

<https://huggingface.co/deepseek-ai>

vLLM model recipes and VRAM formula:

<https://github.com/vllm-project/recipes>

vLLM KV cache layout:

[https://github.com/vllm-](https://github.com/vllm-project/vllm/blob/8eb35c5217ef98ab06c88f42ef4e41737dd2d1f3/vllm/v1/kv_cache_interface.py)

[project/vllm/blob/8eb35c5217ef98ab06c88f42ef4e41737dd2d1f3/vllm/v1/kv_cache_interface.py](https://github.com/vllm-project/vllm/blob/8eb35c5217ef98ab06c88f42ef4e41737dd2d1f3/vllm/v1/kv_cache_interface.py)

vLLM engine arguments:

https://docs.vllm.ai/en/latest/configuration/engine_args/

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
import math
```

```
from dataclasses import asdict, dataclass, field
```

```
from typing import Iterable, Mapping, Sequence
```

```
GB = 1_000_000_000
```

```
@dataclass(frozen=True)
```

```
class Source:
```

```
    """Primary-source provenance for a cited constant."""
```

```
    value: object
```

```
    url: str
```

```

    field: str
    retrieved: str
    derivable: bool = False
    note: str = ""

FLASH_DTYPE_CENSUS = {
    "BF16": 1_483_567_488,
    "I64": 2_327_040,
    "F32": 37_741_630,
    "FP8": 6_304_038_912,
    "FP4": 296_352_743_424,
}

PRO_DTYPE_CENSUS = {
    "BF16": 2_816_899_328,
    "I64": 2_327_040,
    "F32": 87_776_414,
    "FP8": 23_169_335_296,
    "FP4": 1_572_763_336_704,
}

R1_DTYPE_CENSUS = {
    "BF16": 3_918_786_560,
    "FP8": 680_571_043_840,
    "F32": 41_555_600,
}

PROVENANCE: dict[str, Source] = {
    "flash.dtype_census": Source(
        FLASH_DTYPE_CENSUS,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-0731",
        "official Hugging Face safetensors dtype summary",
        "2026-08-12",
    ),
    "pro.dtype_census": Source(
        PRO_DTYPE_CENSUS,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro",
        "official Hugging Face safetensors dtype summary",
        "2026-08-12",
    ),
    "r1.dtype_census": Source(
        R1_DTYPE_CENSUS,
        "https://huggingface.co/deepseek-ai/DeepSeek-R1",
        "official Hugging Face safetensors dtype summary",
        "2026-08-12",
    ),
    "flash.checkpoint_tensor_gb": Source(
        166.878536440,

```

```

        "https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-0731",
        "model.safetensors.index.json metadata.total_size",
        "2026-08-12",
    ),
    "pro.checkpoint_tensor_gb": Source(
        864.704792696,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro",
        "model.safetensors.index.json metadata.total_size",
        "2026-08-12",
    ),
    "r1.checkpoint_tensor_gb": Source(
        688.574839360,
        "https://huggingface.co/deepseek-ai/DeepSeek-R1",
        "HF dtype summary: BF16 + FP8 payload; F32 metadata proxy",
        "2026-08-12",
    ),
    "flash.recipe_vram_hint_gb": Source(
        200.0,
        "https://github.com/vllm-
project/recipes/blob/93683962d28f2ed140c08030e2b9be0a50f646a0/models/deepseek-
ai/DeepSeek-V4-Flash.yaml",
        "variants.default.vram_minimum_gb",
        "2026-08-12",
    ),
    "pro.recipe_vram_hint_gb": Source(
        960.0,
        "https://github.com/vllm-
project/recipes/blob/93683962d28f2ed140c08030e2b9be0a50f646a0/models/deepseek-
ai/DeepSeek-V4-Pro.yaml",
        "variants.default.vram_minimum_gb",
        "2026-08-12",
    ),
    "r1.recipe_vram_hint_gb": Source(
        805.0,
        "https://github.com/vllm-
project/recipes/blob/93683962d28f2ed140c08030e2b9be0a50f646a0/models/deepseek-
ai/DeepSeek-R1.yaml",
        "literal variants.default.vram_minimum_gb (the authoring guide's ceil example
conflicts with arithmetic)",
        "2026-08-13",
        note="Recipe stores 805; ceil(671 * 1 * 1.2) is 806. Treat as an ambiguous
hint, not a derivation.",
    ),
    "flash.context_limit": Source(
        1_048_576,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-
0731/blob/7872f01b1d1fe23eabc4c98b48bfffce5a386062/config.json",
        "max_position_embeddings",

```

```

        "2026-08-12",
    ),
    "pro.context_limit": Source(
        1_048_576,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-
Pro/blob/b5968e9190ef611bbf34a7229255be88a0e937c1/config.json",
        "max_position_embeddings",
        "2026-08-12",
    ),
    "r1.context_limit": Source(
        163_840,
        "https://huggingface.co/deepseek-ai/DeepSeek-
R1/blob/56d4cbbb4d29f4355bab4b9a39ccb717a14ad5ad/config.json",
        "max_position_embeddings",
        "2026-08-12",
    ),
    "flash.layers_c4": Source(
        21,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-
0731/blob/7872f01b1d1fe23eabc4c98b48bfffcef5a386062/config.json",
        "compress_ratios: count(value == 4)",
        "2026-08-12",
    ),
    "flash.layers_c128": Source(
        20,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-
0731/blob/7872f01b1d1fe23eabc4c98b48bfffcef5a386062/config.json",
        "compress_ratios: count(value == 128)",
        "2026-08-12",
    ),
    "pro.layers_c4": Source(
        30,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-
Pro/blob/b5968e9190ef611bbf34a7229255be88a0e937c1/config.json",
        "compress_ratios: count(value == 4)",
        "2026-08-12",
    ),
    "pro.layers_c128": Source(
        31,
        "https://huggingface.co/deepseek-ai/DeepSeek-V4-
Pro/blob/b5968e9190ef611bbf34a7229255be88a0e937c1/config.json",
        "compress_ratios: count(value == 128)",
        "2026-08-12",
    ),
    "r1.mla_layers": Source(
        61,
        "https://huggingface.co/deepseek-ai/DeepSeek-
R1/blob/56d4cbbb4d29f4355bab4b9a39ccb717a14ad5ad/config.json",

```

```

        "num_hidden_layers",
        "2026-08-12",
    ),
    "workspace.max_num_batched_tokens": Source(
        16_384,
        "https://github.com/vllm-
project/recipes/blob/93683962d28f2ed140c08030e2b9be0a50f646a0/models/deepseek-
ai/DeepSeek-V4-Flash.yaml",
        "single_node_dep example --max-num-batched-tokens",
        "2026-08-12",
    ),
    "v4.kv_entry_bytes": Source(
        584,
        "https://github.com/vllm-
project/vllm/blob/8eb35c5217ef98ab06c88f42ef4e41737dd2d1f3/vllm/v1/kv_cache_interface.
py",
        "MLAAttentionSpec.real_page_size_bytes: DeepSeek V4 fp8_ds_mla 448 + 128 + 8",
        "2026-08-12",
    ),
    "nvidia.fp4_native_architecture": Source(
        "blackwell",
        "https://www.nvidia.com/en-sg/data-center/tensor-cores/",
        "Blackwell/Hopper Tensor Core precision support table",
        "2026-08-12",
    ),
    "hardware.h20_141_gb": Source(
        141.0,
        "https://docs.nvidia.com/ai-enterprise/release-8/latest/infra-
software/vgpu/reference/hopper.html",
        "Table 90: NVIDIA H20 141GB HBM3",
        "2026-08-12",
    ),
    "hardware.h20_96_gb": Source(
        96.0,
        "https://docs.nvidia.com/ai-enterprise/release-8/latest/infra-
software/vgpu/reference/hopper.html",
        "Table 93: NVIDIA H20 96GB HBM3",
        "2026-08-12",
    ),
    "hardware.ascend_a2_64_gb": Source(
        64.0,
        "https://docs.vllm.ai/projects/ascend/en/main/tutorials/models/DeepSeek-V4-
Flash.html",
        "Atlas 800 A2 example device memory capacity",
        "2026-08-12",
    ),
    "vllm.hopper_packed_checkpoint": Source(
        "packed mixed FP4/FP8",

```

```

        "https://github.com/vllm-
project/recipes/blob/93683962d28f2ed140c08030e2b9be0a50f646a0/models/deepseek-
ai/DeepSeek-V4-Flash.yaml",
        "H200 verified; default checkpoint vram_minimum_gb=200",
        "2026-08-12",
        note=(
            "This supports packed persistent residency on Hopper. It does not "
            "claim native FP4 arithmetic; compute kernels may dequantize on demand."
        ),
    ),
    "ascend.flash.topology": Source(
        "1 node x 8 A2 NPUs",
        "https://docs.vllm.ai/projects/ascend/en/main/tutorials/models/DeepSeek-V4-
Flash.html",
        "DeepSeek-V4-Flash-0731-w8a8 A2 launch; DP4/TP4/EP; utilization 0.9",
        "2026-08-13",
    ),
    "ascend.pro.topology": Source(
        "4 nodes x 8 A2 NPUs",
        "https://docs.vllm.ai/projects/ascend/en/main/tutorials/models/DeepSeek-V4-
Pro.html",
        "DeepSeek-V4-Pro-w4a8-mtp launch topology; utilization 0.9",
        "2026-08-12",
    ),
    "ascend.r1.topology": Source(
        "2 nodes x 8 A2 NPUs",
        "https://docs.vllm.ai/projects/ascend/en/main/tutorials/models/DeepSeek-
R1.html",
        "DeepSeek-R1-W8A8 launch; DP4/TP4/EP; utilization 0.92",
        "2026-08-13",
    ),
}

DEFAULT_MAX_NUM_BATCHED_TOKENS = int(
    PROVENANCE["workspace.max_num_batched_tokens"].value
)
V4_KV_ENTRY_BYTES = int(PROVENANCE["v4.kv_entry_bytes"].value)

DTYPE_DISK_BYTES: dict[str, float] = {
    "BF16": 2.0,
    "I64": 8.0,
    "F32": 4.0,
    "FP8": 1.0,
    "FP4": 0.5,
}

```



```

@dataclass(frozen=True)
class Hardware:
    key: str
    label: str
    memory_gb: float
    devices_per_node: int
    platform: str
    architecture: str

    @property
    def raw_node_gb(self) -> float:
        return self.memory_gb * self.devices_per_node

```

```

@dataclass(frozen=True)
class ArtifactProfile:
    checkpoint: str
    checkpoint_revision: str | None
    dtype_census: Mapping[str, int] | None
    checkpoint_total_bytes: int | None
    metadata_dtypes: frozenset[str] = frozenset()
    expert_dtype: str | None = None
    documented_utilization: float | None = None

    @property
    def payload_bytes(self) -> int | None:
        if self.dtype_census is None:
            return None
        return int(
            sum(
                count * DTYPE_DISK_BYTES[dtype]
                for dtype, count in self.dtype_census.items()
                if dtype not in self.metadata_dtypes
            )
        )

    @property
    def quant_metadata_bytes(self) -> int | None:
        if self.dtype_census is None:
            return None
        explicit = int(
            sum(
                count * DTYPE_DISK_BYTES[dtype]
                for dtype, count in self.dtype_census.items()
                if dtype in self.metadata_dtypes
            )
        )
        if self.checkpoint_total_bytes is None:

```

```

        return explicit
    payload = self.payload_bytes
    assert payload is not None
    return self.checkpoint_total_bytes - payload

@property
def checkpoint_disk_bytes(self) -> int | None:
    if self.checkpoint_total_bytes is not None:
        return self.checkpoint_total_bytes
    payload = self.payload_bytes
    metadata = self.quant_metadata_bytes
    if payload is None or metadata is None:
        return None
    return payload + metadata

@dataclass(frozen=True)
class ModelProfile:
    key: str
    label: str
    artifacts: Mapping[str, ArtifactProfile]
    recipe_vram_hint_gb: float
    context_limit: int
    kv_kind: str
    layers_c4: int = 0
    layers_c128: int = 0
    layers_swa: int = 0
    r1_mla_layers: int = 0
    documented_ascend_nodes: int = 0

    @property
    def nvidia_artifact(self) -> ArtifactProfile:
        return self.artifacts["nvidia"]

    @property
    def checkpoint(self) -> str:
        return self.nvidia_artifact.checkpoint

    @property
    def checkpoint_revision(self) -> str:
        return self.nvidia_artifact.checkpoint_revision or "unversioned"

    @property
    def dtype_census(self) -> Mapping[str, int]:
        census = self.nvidia_artifact.dtype_census
        assert census is not None
        return census

```

```

@property
def payload_bytes(self) -> int:
    value = self.nvidia_artifact.payload_bytes
    assert value is not None
    return value

@property
def quant_metadata_bytes(self) -> int:
    value = self.nvidia_artifact.quant_metadata_bytes
    assert value is not None
    return value

@property
def payload_gb(self) -> float:
    return self.payload_bytes / GB

@property
def quant_metadata_gb(self) -> float:
    return self.quant_metadata_bytes / GB

@property
def checkpoint_tensor_gb(self) -> float:
    value = self.nvidia_artifact.checkpoint_disk_bytes
    assert value is not None
    return value / GB

@dataclass(frozen=True)
class Envelope:
    concurrent_sequences: int
    average_resident_tokens: int
    block_size: int
    indexer_bytes_per_entry: int

    @property
    def rounded_tokens_per_sequence(self) -> int:
        return math.ceil(self.average_resident_tokens / self.block_size) *
self.block_size

    @property
    def total_rounded_resident_tokens(self) -> int:
        return self.concurrent_sequences * self.rounded_tokens_per_sequence

@dataclass(frozen=True)
class Assumptions:
    runtime_workspace_fraction: float
    graph_comm_fraction: float

```

```
safety_reserve_fraction: float
runtime_workspace_overrides: dict[str, float]
graph_comm_overrides: dict[str, float]
safety_reserve_overrides: dict[str, float]
max_num_batched_tokens: int = DEFAULT_MAX_NUM_BATCHED_TOKENS
workspace_bytes_per_batched_token: float | None = None
workspace_mode: str = "batched_tokens"
fp4_resident_bytes_per_element: float = 0.5
parallelism: str = "tp"
data_parallel_size: int = 1
r1_kv_cache_dtype: str = "auto"
```

```
@dataclass(frozen=True)
```

```
class HBMResult:
```

```
    model: str
    checkpoint: str
    checkpoint_disk_gb: float
    resident_model_weights_gb: float
    quantization_metadata_gb: float
    runtime_model_workspace_gb: float
    kv_cache_gb: float
    graph_communication_buffers_gb: float
    fragmentation_safety_reserve_gb: float
    component_total_gb: float
    recipe_vram_hint_gb: float
    scenario_hbm_gb: float
    per_tp8_rank_average_gb: float
    recipe_hint_delta_percent: float
    recipe_hint_comparison: str
    architecture: str
    resident_weight_policy: str
    parallelism: str
    data_parallel_size: int
    workspace_basis: str
    workspace_bytes_per_batched_token: float | None
    evidence_status: str
```

```
@dataclass(frozen=True)
```

```
class HardwareResult:
```

```
    model: str
    hardware: str
    scenario_hbm_gb: float | None
    utilization: float
    usable_gb_per_device: float | None
    aggregate_capacity_lower_bound_devices: int | None
    allocated_devices: int
```

```

allocated_nodes: int
installed_raw_hbm_gb: float
allocated_executor_budget_gb: float | None
executor_headroom_gb: float | None
aggregate_tp8_budget_covers_scenario: bool | None
current_tp8_fits: bool | None
per_allocated_rank_average_gb: float | None
artifact: str
status: str

```

```

HARDWARE = {
    "h20-141": Hardware(
        "h20-141",
        "NVIDIA H20 141 GB",
        float(PROVENANCE["hardware.h20_141_gb"].value),
        8,
        "nvidia",
        "hopper",
    ),
    "h20-96": Hardware(
        "h20-96",
        "NVIDIA H20 96 GB",
        float(PROVENANCE["hardware.h20_96_gb"].value),
        8,
        "nvidia",
        "hopper",
    ),
    "910b-64": Hardware(
        "910b-64",
        "Huawei Ascend 910B 64 GB",
        float(PROVENANCE["hardware.ascend_a2_64_gb"].value),
        8,
        "ascend",
        "ascend_a2",
    ),
}

```

```

# V4 payload bytes are DERIVED from the official Hugging Face dtype census and
# DTYPE_DISK_BYTES. The residual to the cited safetensors total is retained as
# block-scale/packing metadata. R1's old index reports an unquantized total, so
# its official dtype summary is used: BF16 + FP8 payload and F32 metadata proxy.

```

```

MODELS = {
    "flash-0731": ModelProfile(
        key="flash-0731",
        label="DeepSeek V4 Flash 0731",
        artifacts={

```

```

        "nvidia": ArtifactProfile(
            checkpoint="deepseek-ai/DeepSeek-V4-Flash-0731",
            checkpoint_revision="7872f01b1d1fe23eabc4c98b48bffcef5a386062",
            dtype_census=FLASH_DTYPE_CENSUS,
            checkpoint_total_bytes=166_878_536_440,
            expert_dtype="FP4",
        ),
        "ascend": ArtifactProfile(
            checkpoint="vllm-ascend/DeepSeek-V4-Flash-0731-w8a8",
            checkpoint_revision=None,
            dtype_census=None,
            checkpoint_total_bytes=None,
            documented_utilization=0.90,
        ),
    },
    recipe_vram_hint_gb=float(
        PROVENANCE["flash.recipe_vram_hint_gb"].value
    ),
    context_limit=int(PROVENANCE["flash.context_limit"].value),
    kv_kind="v4",
    layers_c4=int(PROVENANCE["flash.layers_c4"].value),
    layers_c128=int(PROVENANCE["flash.layers_c128"].value),
    layers_swa=2,
    documented_ascend_nodes=1,
),
"v4-pro": ModelProfile(
    key="v4-pro",
    label="DeepSeek V4 Pro",
    artifacts={
        "nvidia": ArtifactProfile(
            checkpoint="deepseek-ai/DeepSeek-V4-Pro",
            checkpoint_revision="b5968e9190ef611bbf34a7229255be88a0e937c1",
            dtype_census=PRO_DTYPE_CENSUS,
            checkpoint_total_bytes=864_704_792_696,
            expert_dtype="FP4",
        ),
        "ascend": ArtifactProfile(
            checkpoint="vllm-ascend/DeepSeek-V4-Pro-w4a8-mtp",
            checkpoint_revision=None,
            dtype_census=None,
            checkpoint_total_bytes=None,
            documented_utilization=0.90,
        ),
    },
    recipe_vram_hint_gb=float(
        PROVENANCE["pro.recipe_vram_hint_gb"].value
    ),
    context_limit=int(PROVENANCE["pro.context_limit"].value),

```

```

        kv_kind="v4",
        layers_c4=int(PROVENANCE["pro.layers_c4"].value),
        layers_c128=int(PROVENANCE["pro.layers_c128"].value),
        documented_ascend_nodes=4,
    ),
    "r1-671b": ModelProfile(
        key="r1-671b",
        label="DeepSeek R1 671B",
        artifacts={
            "nvidia": ArtifactProfile(
                checkpoint="deepseek-ai/DeepSeek-R1",
                checkpoint_revision="56d4cbbb4d29f4355bab4b9a39ccb717a14ad5ad",
                dtype_census=R1_DTYPE_CENSUS,
                checkpoint_total_bytes=None,
                metadata_dtypes=frozenset({"F32"}),
            ),
            "ascend": ArtifactProfile(
                checkpoint="vllm-ascend/DeepSeek-R1-W8A8",
                checkpoint_revision=None,
                dtype_census=None,
                checkpoint_total_bytes=None,
                documented_utilization=0.92,
            ),
        },
        recipe_vram_hint_gb=float(
            PROVENANCE["r1.recipe_vram_hint_gb"].value
        ),
        context_limit=int(PROVENANCE["r1.context_limit"].value),
        kv_kind="r1_mla",
        r1_mla_layers=int(PROVENANCE["r1.mla_layers"].value),
        documented_ascend_nodes=2,
    ),
}

# SIMULATED continuity coefficients, DERIVED by back-fitting the former 3% of
# checkpoint workspace at the CITED 16,384-token recipe default. They are kept
# per model because one global coefficient cannot reproduce three architectures.
DEFAULT_WORKSPACE_BYTES_PER_BATCHED_TOKEN = {
    key: model.checkpoint_tensor_gb * 0.03 * GB / DEFAULT_MAX_NUM_BATCHED_TOKENS
    for key, model in MODELS.items()
}

def validate_fraction(name: str, value: float, *, allow_zero: bool = True) -> None:
    lower_ok = value >= 0 if allow_zero else value > 0
    if not lower_ok or value >= 1:
        left = "[0" if allow_zero else "(0"

```

```

        raise ValueError(f"{name} must be in {left}, 1), got {value}")

def parse_overrides(values: Sequence[str], name: str) -> dict[str, float]:
    result: dict[str, float] = {}
    for value in values:
        try:
            model_key, raw = value.split("=", 1)
            amount = float(raw)
        except ValueError as exc:
            raise ValueError(f"{name} must use MODEL=GB, got {value!r}") from exc
        if model_key not in MODELS:
            raise ValueError(f"unknown model in {name}: {model_key}")
        if amount < 0:
            raise ValueError(f"{name} GB must be non-negative")
        result[model_key] = amount
    return result

def kv_cache_gb(
    model: ModelProfile,
    envelope: Envelope,
    r1_kv_cache_dtype: str = "auto",
) -> float:
    if envelope.concurrent_sequences <= 0:
        raise ValueError("concurrent sequences must be positive")
    if envelope.average_resident_tokens <= 0:
        raise ValueError("average resident tokens must be positive")
    if envelope.block_size <= 0:
        raise ValueError("block size must be positive")
    if envelope.average_resident_tokens > model.context_limit:
        raise ValueError(f"{model.label}: average resident tokens exceed context
limit")

    tokens = envelope.rounded_tokens_per_sequence
    if model.kv_kind == "r1_mla":
        if r1_kv_cache_dtype not in {"auto", "fp8", "bf16"}:
            raise ValueError("R1 KV cache dtype must be auto, fp8, or bf16")
        resolved_dtype = "bf16" if r1_kv_cache_dtype == "auto" else r1_kv_cache_dtype
        bytes_per_element = {"fp8": 1, "bf16": 2}[resolved_dtype]
        # The default recipe leaves cache dtype at auto; vLLM then uses the
        # pinned model's BF16 dtype. FP8 is an explicit alternative scenario.
        per_sequence = (
            tokens
            * model.r1_mla_layers
            * (512 + 64)
            * bytes_per_element
        )

```



```

        return envelope.concurrent_sequences * per_sequence / GB

# Current vLLM DeepSeek-V4 custom FP8 entry:
# 448 FP8 bytes + 128 BF16 bytes + 8 scale bytes = 584 bytes.
main_entry = V4_KV_ENTRY_BYTES
c4_entries = math.ceil(tokens / 4)
c128_entries = math.ceil(tokens / 128)
dynamic = (
    model.layers_c4
    * c4_entries
    * (main_entry + envelope.indexer_bytes_per_entry)
    + model.layers_c128 * c128_entries * main_entry
)
# Every compressed layer retains a 128-token uncompressed shared-KV sliding
# window. C4 additionally retains an 8-entry indexer compressor residual.
# Flash also has two pure sliding-window layers. This is an
# implementation-informed estimate; vLLM page-pool alignment can allocate
# more than the logical state counted here.
fixed = (
    model.layers_c4
    * (128 * main_entry + 8 * envelope.indexer_bytes_per_entry)
    + model.layers_c128 * 128 * main_entry
    + model.layers_swa * 128 * main_entry
)
return envelope.concurrent_sequences * (dynamic + fixed) / GB

def resolve_resident_weights(
    model: ModelProfile,
    assumptions: Assumptions,
    architecture: str,
) -> tuple[float, float, str]:
    """Return resident payload GB, resident metadata GB, and policy label.

    DERIVED: non-expert groups use their cited census and declared logical disk
    widths. V4 FP4 is the expert proxy because the official checkpoint summary
    does not publish a separate expert/dense census. SIMULATED: the resident FP4
    width is explicit and CLI-overridable. Packed 0.5 B is the default because
    official vLLM recipes validate the mixed checkpoint on Hopper; 1 B and 2 B
    model materialized FP8 and BF16 expert copies respectively.
    """
    artifact = model.nvidia_artifact
    census = artifact.dtype_census
    assert census is not None

    if assumptions.parallelism not in {"tp", "dp"}:
        raise ValueError("parallelism must be 'tp' or 'dp'")
    if assumptions.data_parallel_size <= 0:

```

```

        raise ValueError("data parallel size must be positive")
    if assumptions.fp4_resident_bytes_per_element <= 0:
        raise ValueError("FP4 resident bytes per element must be positive")

    dp = assumptions.data_parallel_size if assumptions.parallelism == "dp" else 1
    expert_dtype = artifact.expert_dtype
    if dp > 1 and expert_dtype is None:
        raise ValueError(
            f"{model.label}: DP aggregate residency requires a published "
            "expert/dense split; this checkpoint's dtype census does not provide one"
        )
    if dp > 1 and expert_dtype == "FP4":
        raise ValueError(
            f"{model.label}: exact DP residency requires tensor-name expert/dense "
            "classification; the checkpoint API's dtype census is insufficient"
        )

    dense_bytes = 0.0
    expert_bytes = 0.0
    for dtype, count in census.items():
        if dtype in artifact.metadata_dtypes:
            continue
        if dtype == expert_dtype:
            expert_bytes += count * assumptions.fp4_resident_bytes_per_element
        else:
            dense_bytes += count * DTYPE_DISK_BYTES[dtype]

    resident_payload_bytes = dense_bytes * dp + expert_bytes
    metadata_bytes = artifact.quant_metadata_bytes or 0
    if expert_dtype == "FP4" and assumptions.fp4_resident_bytes_per_element > 0.5:
        # SIMULATED expanded copies fold block scales into materialized values;
        # do not add the checkpoint's packed FP4 scale bytes a second time.
        metadata_bytes = 0

    if expert_dtype == "FP4":
        width = assumptions.fp4_resident_bytes_per_element
        if width == 0.5:
            storage = "packed FP4"
        elif width == 1.0:
            storage = "simulated materialized FP8"
        elif width == 2.0:
            storage = "simulated materialized BF16"
        else:
            storage = f"simulated FP4 expert width {width:g} B/element"
        policy = f"{architecture}; {storage}; dense replication x{dp}"
    else:
        policy = f"{architecture}; cited checkpoint widths; dense replication x{dp}"

```

```

    return resident_payload_bytes / GB, metadata_bytes / GB, policy

def workspace_term(
    model: ModelProfile, assumptions: Assumptions
) -> tuple[float, str, float | None]:
    if model.key in assumptions.runtime_workspace_overrides:
        return (
            assumptions.runtime_workspace_overrides[model.key],
            "measured/explicit per-model GB override",
            None,
        )
    if assumptions.max_num_batched_tokens <= 0:
        raise ValueError("max_num_batched_tokens must be positive")
    if assumptions.workspace_mode == "weights":
        return (
            model.checkpoint_tensor_gb * assumptions.runtime_workspace_fraction,
            "legacy simulated fraction of checkpoint disk GB",
            None,
        )
    if assumptions.workspace_mode != "batched_tokens":
        raise ValueError("workspace mode must be 'batched_tokens' or 'weights'")

    coefficient = assumptions.workspace_bytes_per_batched_token
    if coefficient is None:
        # Backward-compatible zeroing: the old API used a zero fraction to remove
        # every simulated term. Otherwise use the per-model continuity mapping.
        coefficient = (
            0.0
            if assumptions.runtime_workspace_fraction == 0.0
            else DEFAULT_WORKSPACE_BYTES_PER_BATCHED_TOKEN[model.key]
        )
        basis = "simulated back-fitted bytes per batched token"
    else:
        if coefficient < 0:
            raise ValueError("workspace bytes per batched token must be non-negative")
        basis = "explicit simulated bytes per batched token"
    return coefficient * assumptions.max_num_batched_tokens / GB, basis, coefficient

def size_model(
    model: ModelProfile,
    envelope: Envelope,
    assumptions: Assumptions,
    architecture: str = "hopper",
) -> HBMResult:
    resident_weights, resident_metadata, resident_policy = resolve_resident_weights(
        model, assumptions, architecture
    )

```

```

)
workspace, workspace_basis, workspace_coefficient = workspace_term(
    model, assumptions
)
graph_comm = assumptions.graph_comm_overrides.get(
    model.key, model.checkpoint_tensor_gb * assumptions.graph_comm_fraction
)
kv_gb = kv_cache_gb(model, envelope, assumptions.r1_kv_cache_dtype)
pre_reserve = (
    resident_weights
    + resident_metadata
    + workspace
    + kv_gb
    + graph_comm
)
reserve = assumptions.safety_reserve_overrides.get(
    model.key,
    pre_reserve * assumptions.safety_reserve_fraction,
)
component_total = pre_reserve + reserve
# The recipe value is a parameter-count authoring heuristic with an
# undefined 20% margin, not a measured runtime floor. Keep it as a
# comparison and never combine it algebraically with scenario components.
recipe_hint_delta_percent = (
    (component_total - model.recipe_vram_hint_gb)
    / model.recipe_vram_hint_gb
    * 100
)
if recipe_hint_delta_percent < 0:
    recipe_hint_comparison = (
        f"recipe hint exceeds scenario components by {-
recipe_hint_delta_percent:.1f}%"
    )
else:
    recipe_hint_comparison = (
        f"scenario components exceed recipe hint by
{recipe_hint_delta_percent:.1f}%"
    )

overridden = any(
    model.key in values
    for values in (
        assumptions.runtime_workspace_overrides,
        assumptions.graph_comm_overrides,
        assumptions.safety_reserve_overrides,
    )
)
fp4_simulated = assumptions.fp4_resident_bytes_per_element != 0.5

```

```

evidence_status = (
    "mixed: checkpoint/KV cited or derived; workspace and reserves simulated"
    + ("; FP4 resident width simulated" if fp4_simulated else "")
    + ("; one or more runtime terms overridden" if overridden else "")
)
return HBMResult(
    model=model.label,
    checkpoint=model.checkpoint,
    checkpoint_disk_gb=model.checkpoint_tensor_gb,
    resident_model_weights_gb=resident_weights,
    quantization_metadata_gb=resident_metadata,
    runtime_model_workspace_gb=workspace,
    kv_cache_gb=kv_gb,
    graph_communication_buffers_gb=graph_comm,
    fragmentation_safety_reserve_gb=reserve,
    component_total_gb=component_total,
    recipe_vram_hint_gb=model.recipe_vram_hint_gb,
    scenario_hbm_gb=component_total,
    # Deprecated compatibility field; allocation-specific averages are in
    # HardwareResult.per_allocated_rank_average_gb.
    per_tp8_rank_average_gb=component_total / 8,
    recipe_hint_delta_percent=recipe_hint_delta_percent,
    recipe_hint_comparison=recipe_hint_comparison,
    architecture=architecture,
    resident_weight_policy=resident_policy,
    parallelism=assumptions.parallelism,
    data_parallel_size=assumptions.data_parallel_size,
    workspace_basis=workspace_basis,
    workspace_bytes_per_batched_token=workspace_coefficient,
    evidence_status=evidence_status,
)

def hardware_size(
    model: ModelProfile,
    result: HBMResult,
    hardware: Hardware,
    utilization: float,
) -> HardwareResult:
    artifact = model.artifacts[hardware.platform]
    if artifact.checkpoint_disk_bytes is None:
        nodes = model.documented_ascend_nodes
        devices = nodes * hardware.devices_per_node
        documented_utilization = artifact.documented_utilization or utilization
    return HardwareResult(
        model=model.label,
        hardware=hardware.label,
        scenario_hbm_gb=None,

```

```

        utilization=documented_utilization,
        usable_gb_per_device=None,
        aggregate_capacity_lower_bound_devices=None,
        allocated_devices=devices,
        allocated_nodes=nodes,
        installed_raw_hbm_gb=nodes * hardware.raw_node_gb,
        allocated_executor_budget_gb=None,
        executor_headroom_gb=None,
        aggregate_tp8_budget_covers_scenario=None,
        current_tp8_fits=None,
        per_allocated_rank_average_gb=None,
        artifact=artifact.checkpoint,
        status=(
            "artifact memory unknown: no published dtype census; official "
            f"topology is {nodes} x {hardware.devices_per_node} devices at "
            f"utilization {documented_utilization:.2f}"
        ),
    ),
)

usable_device = hardware.memory_gb * utilization
capacity_lower_bound = math.ceil(result.scenario_hbm_gb / usable_device)
nodes = max(1, math.ceil(capacity_lower_bound / hardware.devices_per_node))
devices = nodes * hardware.devices_per_node
raw = nodes * hardware.raw_node_gb
budget = raw * utilization
aggregate_tp8_covers = result.scenario_hbm_gb <= 8 * usable_device
status = (
    "UNVERIFIED aggregate-capacity scenario only; H20 is not recipe-verified "
    "and per-rank TP/DP/EP/PP residency was not modeled"
)

return HardwareResult(
    model=model.label,
    hardware=hardware.label,
    scenario_hbm_gb=result.scenario_hbm_gb,
    utilization=utilization,
    usable_gb_per_device=usable_device,
    aggregate_capacity_lower_bound_devices=capacity_lower_bound,
    allocated_devices=devices,
    allocated_nodes=nodes,
    installed_raw_hbm_gb=raw,
    allocated_executor_budget_gb=budget,
    executor_headroom_gb=budget - result.scenario_hbm_gb,
    aggregate_tp8_budget_covers_scenario=aggregate_tp8_covers,
    current_tp8_fits=None,
    per_allocated_rank_average_gb=result.scenario_hbm_gb / devices,
    artifact=artifact.checkpoint,
    status=status,
)

```

```

)

def fmt(value: float | None) -> str:
    return "unknown" if value is None else f"{value:,.1f}"

def print_table(headers: Sequence[str], rows: Iterable[Sequence[str]]) -> None:
    rows_list = [list(row) for row in rows]
    widths = [len(header) for header in headers]
    for row in rows_list:
        for index, value in enumerate(row):
            widths[index] = max(widths[index], len(value))
    print(" ".join(header.ljust(widths[i]) for i, header in enumerate(headers)))
    print(" ".join("-" * width for width in widths))
    for row in rows_list:
        print(" ".join(value.ljust(widths[i]) for i, value in enumerate(row)))

def print_summary(results: Sequence[HBMResult], envelope: Envelope) -> None:
    print("DeepSeek GPU HBM scenario for one model instance (decimal GB)")
    print(
        f"Envelope: {envelope.concurrent_sequences} resident sequences x "
        f"{envelope.average_resident_tokens:,} average tokens "
        f"(rounded to {envelope.block_size}-token blocks)"
    )
    print(
        "Runtime terms and non-default FP4 resident widths are simulated; "
        "replace with measured values before approval.\n"
    )
    print_table(
        ("Model", "Disk GB", "Resident weights GB", "Scenario GB", "Recipe hint GB",
"Comparison"),
        (
            (
                result.model,
                fmt(result.checkpoint_disk_gb),
                fmt(result.resident_model_weights_gb),
                fmt(result.scenario_hbm_gb),
                fmt(result.recipe_vram_hint_gb),
                result.recipe_hint_comparison,
            )
            for result in results
        ),
    )
    print(
        "\nWorkspace default: model-specific bytes/batched-token coefficients "
        "back-fitted to the former 3% estimates at 16,384 tokens (SIMULATED)."
    )

```

```

)

def print_components(results: Sequence[HBMResult]) -> None:
    print("HBM component breakdown (decimal GB)\n")
    print_table(
        ("Model", "Disk", "Resident weights", "Quant meta", "Workspace", "KV",
"Graph/comm", "Reserve", "Scenario"),
        (
            (
                result.model,
                fmt(result.checkpoint_disk_gb),
                fmt(result.resident_model_weights_gb),
                fmt(result.quantization_metadata_gb),
                fmt(result.runtime_model_workspace_gb),
                fmt(result.kv_cache_gb),
                fmt(result.graph_communication_buffers_gb),
                fmt(result.fragmentation_safety_reserve_gb),
                fmt(result.scenario_hbm_gb),
            )
            for result in results
        ),
    )
    print("\nEvidence and policy:")
    for result in results:
        coefficient = (
            "n/a"
            if result.workspace_bytes_per_batched_token is None
            else f"{result.workspace_bytes_per_batched_token:,.0f} B/batched-token"
        )
        print(
            f"- {result.model}: {result.resident_weight_policy}; workspace="
            f"{result.workspace_basis} ({coefficient});"
            f"{result.recipe_hint_comparison}."
        )

def print_hardware(results: Sequence[HardwareResult]) -> None:
    print("Hardware conversion and cited Ascend topology (decimal GB)\n")
    print_table(
        ("Model", "Hardware", "Scenario", "Usable/device", "Capacity LB", "Allocated",
"GB/allocated rank", "TP8 aggregate covers"),
        (
            (
                result.model,
                result.hardware,
                fmt(result.scenario_hbm_gb),
                fmt(result.usable_gb_per_device),

```



```

        (
            "unknown"
            if result.aggregate_capacity_lower_bound_devices is None
            else str(result.aggregate_capacity_lower_bound_devices)
        ),
        f"{result.allocated_devices} ({result.allocated_nodes} node{'s' if
result.allocated_nodes != 1 else ''})",
        fmt(result.per_allocated_rank_average_gb),
        (
            "unknown"
            if result.aggregate_tp8_budget_covers_scenario is None
            else "yes" if result.aggregate_tp8_budget_covers_scenario else
"no"
        ),
    ),
    for result in results
),
)
print(
    "\nCapacity LB is aggregate only and does not prove a runnable per-rank
topology. "
    "Ascend memory is unknown because its separate artifact has no published dtype
census."
)

def parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(
        description="Calculate per-model GPU HBM in decimal GB; no throughput or
replica sizing.",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
    )
    p.add_argument("--models", nargs="+", choices=MODELS, default=list(MODELS))
    p.add_argument("--hardware", nargs="+", choices=HARDWARE, default=list(HARDWARE))
    p.add_argument("--view", choices=("summary", "components", "hardware", "all"),
default="summary")
    p.add_argument("--format", choices=("table", "json"), default="table")
    p.add_argument("--concurrent-sequences", type=int, default=256)
    p.add_argument("--average-resident-tokens", type=int, default=4096)
    p.add_argument("--block-size", type=int, default=256)
    p.add_argument(
        "--indexer-bytes-per-entry",
        type=int,
        default=256,
        help="V4 sparse-indexer cache bytes per compressed entry; BF16 default for
Hopper",
    )
    p.add_argument(

```

```

        "--r1-kv-cache-dtype",
        choices=("auto", "fp8", "bf16"),
        default="auto",
        help="auto follows the pinned R1 model dtype (BF16); FP8 is explicit",
    )
    p.add_argument("--gpu-memory-utilization", type=float, default=0.75)
    p.add_argument(
        "--max-num-batched-tokens",
        type=int,
        default=DEFAULT_MAX_NUM_BATCHED_TOKENS,
        help="scheduler token budget used by the simulated workspace model",
    )
    p.add_argument(
        "--workspace-mode",
        choices=("batched_tokens", "weights"),
        default="batched_tokens",
        help="workspace scaling model; weights is the legacy fallback",
    )
    p.add_argument(
        "--workspace-bytes-per-batched-token",
        type=float,
        default=None,
        help=(
            "simulated coefficient applied to every selected model; when omitted, "
            "a clearly labelled per-model back-fit preserves prior defaults"
        ),
    )
    p.add_argument(
        "--runtime-workspace-fraction",
        type=float,
        default=0.03,
        help="legacy simulated coefficient used only with --workspace-mode weights",
    )
    p.add_argument("--graph-comm-fraction", type=float, default=0.02)
    p.add_argument("--safety-reserve-fraction", type=float, default=0.10)
    p.add_argument(
        "--fp4-resident-bytes-per-element",
        type=float,
        choices=(0.5, 1.0, 2.0),
        default=0.5,
        help=(
            "SIMULATED persistent FP4 expert width: 0.5 packed, 1 materialized "
            "FP8, or 2 materialized BF16"
        ),
    )
    p.add_argument(
        "--parallelism",
        choices=("tp", "dp"),

```

```

        default="tp",
        help="aggregate resident-weight policy",
    )
    p.add_argument(
        "--data-parallel-size",
        type=int,
        default=1,
        help="dense replication degree when --parallelism dp",
    )
    p.add_argument(
        "--runtime-workspace-gb",
        action="append",
        default=[],
        metavar="MODEL=GB",
        help="measured per-model override; may be repeated",
    )
    p.add_argument(
        "--graph-comm-gb",
        action="append",
        default=[],
        metavar="MODEL=GB",
        help="measured per-model override; may be repeated",
    )
    p.add_argument(
        "--safety-reserve-gb",
        action="append",
        default=[],
        metavar="MODEL=GB",
        help="explicit per-model override; may be repeated",
    )
    p.add_argument("--self-test", action="store_true")
    return p

```

```

def self_test() -> None:
    assert abs(MODELS["flash-0731"].checkpoint_tensor_gb - 166.87853644) < 1e-9
    assert abs(MODELS["v4-pro"].checkpoint_tensor_gb - 864.704792696) < 1e-9
    assert abs(MODELS["r1-671b"].checkpoint_tensor_gb - 688.57483936) < 1e-9
    envelope = Envelope(256, 4096, 256, 256)
    assert envelope.total_rounded_resident_tokens == 1_048_576
    flash_kv = kv_cache_gb(MODELS["flash-0731"], envelope)
    pro_kv = kv_cache_gb(MODELS["v4-pro"], envelope)
    r1_kv = kv_cache_gb(MODELS["r1-671b"], envelope)
    assert 5.3 < flash_kv < 5.7
    assert 7.7 < pro_kv < 8.1
    assert 73.5 < r1_kv < 73.9
    assert 36.7 < kv_cache_gb(MODELS["r1-671b"], envelope, "fp8") < 37.0
    assumptions = Assumptions(0.03, 0.02, 0.10, {}, {}, {})

```

```

flash = size_model(MODELS["flash-0731"], envelope, assumptions)
assert abs(flash.checkpoint_disk_gb - 166.87853644) < 1e-9
assert 157.5 < flash.resident_model_weights_gb < 157.7
expanded = size_model(
    MODELS["flash-0731"],
    envelope,
    Assumptions(
        0.03,
        0.02,
        0.10,
        {},
        {},
        {},
        fp4_resident_bytes_per_element=1.0,
    ),
)
assert 305.7 < expanded.resident_model_weights_gb < 305.9
assert expanded.quantization_metadata_gb == 0
print("self-test: ok")

def main() -> None:
    args = parser().parse_args()
    if args.self_test:
        self_test()
        return

    validate_fraction("gpu memory utilization", args.gpu_memory_utilization,
allow_zero=False)
    validate_fraction("runtime workspace fraction", args.runtime_workspace_fraction)
    validate_fraction("graph/communication fraction", args.graph_comm_fraction)
    validate_fraction("safety reserve fraction", args.safety_reserve_fraction)
    if args.indexer_bytes_per_entry < 0:
        raise ValueError("indexer bytes per entry must be non-negative")
    if args.max_num_batched_tokens <= 0:
        raise ValueError("max num batched tokens must be positive")
    if args.data_parallel_size <= 0:
        raise ValueError("data parallel size must be positive")
    if (
        args.workspace_bytes_per_batched_token is not None
        and args.workspace_bytes_per_batched_token < 0
    ):
        raise ValueError("workspace bytes per batched token must be non-negative")

    envelope = Envelope(
        args.concurrent_sequences,
        args.average_resident_tokens,
        args.block_size,

```

```

    args.indexer_bytes_per_entry,
)
assumptions = Assumptions(
    runtime_workspace_fraction=args.runtime_workspace_fraction,
    graph_comm_fraction=args.graph_comm_fraction,
    safety_reserve_fraction=args.safety_reserve_fraction,
    runtime_workspace_overrides=parse_overrides(
        args.runtime_workspace_gb, "runtime workspace override"
    ),
    graph_comm_overrides=parse_overrides(args.graph_comm_gb, "graph/comm
override"),
    safety_reserve_overrides=parse_overrides(
        args.safety_reserve_gb, "safety reserve override"
    ),
    max_num_batched_tokens=args.max_num_batched_tokens,
    workspace_bytes_per_batched_token=args.workspace_bytes_per_batched_token,
    workspace_mode=args.workspace_mode,
    fp4_resident_bytes_per_element=args.fp4_resident_bytes_per_element,
    parallelism=args.parallelism,
    data_parallel_size=args.data_parallel_size,
    r1_kv_cache_dtype=args.r1_kv_cache_dtype,
)
selected_models = [MODELS[key] for key in args.models]
results = [size_model(model, envelope, assumptions) for model in selected_models]
hardware_results = [
    hardware_size(model, result, HARDWARE[hardware_key],
args.gpu_memory_utilization)
    for model, result in zip(selected_models, results)
    for hardware_key in args.hardware
]

if args.format == "json":
    print(
        json.dumps(
            {
                "units": "decimal GB (1 GB = 1,000,000,000 bytes)",
                "objective": "aggregate GPU/NPU HBM for one model instance",
                "envelope": asdict(envelope),
                "assumptions": asdict(assumptions),
                "models": [asdict(result) for result in results],
                "hardware_conversion": [asdict(result) for result in
hardware_results],
                "provenance": {
                    name: asdict(source) for name, source in PROVENANCE.items()
                },
            },
            indent=2,
        )
    )

```

```

    )
    return

if args.view in ("summary", "all"):
    print_summary(results, envelope)
if args.view in ("components", "all"):
    if args.view == "all":
        print()
    print_components(results)
if args.view in ("hardware", "all"):
    if args.view == "all":
        print()
    print_hardware(hardware_results)

if __name__ == "__main__":
    main()

```

Program 2: The Fleet & Workload Sizer

OVERVIEW

This program acts like an infrastructure architect. Instead of just looking at one model in a vacuum, it looks at the real-world demand you expect (your user base) and tells you how much total hardware you need to buy or rent.

- **Purpose:** To perform "evidence-aware sizing" for a fleet of servers. It calculates the minimum number of GPU devices you need to handle a specific workload without bottlenecking.
- **Function:** You feed the script a hypothetical scenario (e.g., "I have 200 users, each making a request per hour, with 4096 input tokens"). The program then:
 - Calculates the required server concurrency and data throughput.
 - Determines if the physical hardware (like an 8-GPU cluster) has enough capacity.

- Checks for software compatibility, flagging warnings or "blockers" if your deployment is attempting to use outdated systems (like older vLLM or CANN versions).
- **Usage:** It is run via the command line using heavy customization. You input your exact user demands (using flags like --users or --input-tokens) and hardware configurations to generate reports on whether your setup will survive the expected traffic.

USAGE EXAMPLES

This tool requires you to define workload profiles (users, tokens) and checks them against hardware constraints.

1. Basic Capacity Check with Illustrative Rates

If you want a quick capacity check using the program's built-in (but simulated) throughput rates:

```
python3 deepseek_deployment_sizing.py --view capacity --illustrative-rates
```

- **What it does:** Uses the --illustrative-rates flag to populate the internal service rates for all hardware/model pairs so it can actually calculate the "Candidate fleet" and "Configured fleet" numbers, outputting them in the capacity view.

2. Custom User Workload with Issue Tracking

If you want to see if your current setup can handle a much larger workload, while surfacing any hardware or versioning warnings:

```
python3 deepseek_deployment_sizing.py --users 500 --input-tokens 8192 --output-tokens 1024 --show-issues
```

- **What it does:** Simulates a workload of 500 users processing large documents (8192 input tokens) and generating long responses (1024 output tokens). The --show-issues flag appends a list of warnings or blockers at the bottom of the output (e.g., warning you about unverified H20 deployments).

3. Inputting Real Benchmark Data

If you ran a physical benchmark on an NVIDIA H20-141 for DeepSeek V4 Pro and want to feed those real measurements into the tool to size your fleet:

```
python3 deepseek_deployment_sizing.py --models v4-pro --hardware h20-141 --service-rate "v4-pro@h20-141=1500,150,MEAS"
```

- **What it does:** Restricts the run to the v4-pro and h20-141. The --service-rate flag injects your real-world benchmark (1500 input tokens/sec, 150 output tokens/sec) tagged as MEAS (measured), overriding the need for illustrative rates.

SOURCE CODE

```
#!/usr/bin/env python3
"""Evidence-aware sizing for DeepSeek V4 Flash 0731, V4 Pro, and R1.

The tool keeps three answers separate:

1. ``capacity_lower_bound_devices``: an aggregate-capacity lower bound covering
   the documented device minimum and the larger of the independent recipe-hint
   and workload-KV constraints, not a runnable topology or validated H20 recipe;
2. ``configured_devices``: the supplied KPMG TP=8 allocation; and
3. fleet counts: calculated only when a model+hardware service rate is supplied,
   or when the user explicitly opts into the built-in illustrative rates.

The default ``--gpu-memory-utilization 0.75`` is the supplied KPMG deployment
policy. Current upstream vLLM defaults to 0.92, but silently replacing a supplied
production setting with a library default would answer a different question.

Replace illustrative rates with repeatable ``vllm bench serve`` results before
procurement. H20 is not H200 and is not treated as recipe-verified hardware.

Primary evidence:
  DeepSeek V4 paper/configs: https://arxiv.org/html/2606.19348
  DeepSeek R1: https://github.com/deepseek-ai/DeepSeek-R1
  vLLM recipes: https://github.com/vllm-project/recipes/tree/main/models/deepseek-ai
  vLLM cache config: https://github.com/vllm-project/vllm/blob/main/vllm/config/cache.py
  vLLM-Ascend guides: https://github.com/vllm-project/vllm-ascend/tree/main/docs/source/tutorials/models
"""

from __future__ import annotations

import argparse
import json
import math
from dataclasses import asdict, dataclass
from typing import Dict, Iterable, List, Mapping, Optional, Sequence, Tuple

from packaging.version import InvalidVersion, Version

GB = 1_000_000_000
```



```
SEVERITY_RANK = {"NOTE": 0, "WARNING": 1, "BLOCKER": 2}
```

```
@dataclass(frozen=True)
class Hardware:
    key: str
    label: str
    runtime_family: str
    memory_gb: float
    devices_per_node: int = 8
```

```
@dataclass(frozen=True)
class ArtifactPlan:
    artifact: str
    recipe_vram_hint_gb: Optional[float]
    hint_basis: str
    documented_devices: int
    topology_basis: str
    min_vllm_version_override: Optional[str] = None
```

```
@dataclass(frozen=True)
class ModelProfile:
    key: str
    label: str
    context_limit: int
    min_vllm_version: Optional[str]
    min_vllm_ascend_version: Optional[str]
    nvidia: ArtifactPlan
    ascend: ArtifactPlan
    v4_c4a_layers: int = 0
    v4_c128a_layers: int = 0
    v4_swa_layers: int = 0
```

```
@dataclass(frozen=True)
class Workload:
    name: str
    users: int
    requests_per_user_hour: float
    peak_factor: float
    input_tokens: int
    output_tokens: int
    mean_residence_seconds: float

    @property
    def peak_requests_per_second(self) -> float:
```

```

        return self.users * self.requests_per_user_hour * self.peak_factor / 3600.0

@property
def input_tps(self) -> float:
    return self.peak_requests_per_second * self.input_tokens

@property
def output_tps(self) -> float:
    return self.peak_requests_per_second * self.output_tokens

@property
def concurrent_requests(self) -> float:
    return self.peak_requests_per_second * self.mean_residence_seconds

@dataclass(frozen=True)
class ServiceRate:
    input_tps: float
    output_tps: float
    provenance: str

@dataclass(frozen=True)
class Issue:
    severity: str
    code: str
    message: str

@dataclass(frozen=True)
class SizingResult:
    model_key: str
    model: str
    hardware_key: str
    hardware: str
    workload: str
    artifact: str
    recipe_vram_hint_gb: Optional[float]
    hint_basis: str
    kv_profile: str
    kv_bytes_per_token: int
    kv_fixed_bytes_per_sequence: int
    padded_resident_tokens: int
    fleet_concurrency: float
    concurrency_per_replica: float
    kv_gb_per_replica: float
    runtime_reserve_gb_per_device: float
    capacity_lower_bound_devices: int

```

```

capacity_lower_bound_nodes: int
capacity_lower_bound_usable_hbm_gb: float
configured_devices: int
configured_usable_hbm_gb: float
configured_required_hbm_gb: Optional[float]
configured_headroom_gb: Optional[float]
configured_memory_fit: Optional[bool]
required_fleet_input_tps: float
required_fleet_output_tps: float
required_input_tps_for_one_replica: Optional[float]
required_output_tps_for_one_replica: Optional[float]
service_input_tps: Optional[float]
service_output_tps: Optional[float]
service_rate_provenance: str
throughput_replicas: Optional[int]
candidate_fleet_devices: Optional[int]
configured_fleet_devices: Optional[int]
warm_standby_candidate_devices: Optional[int]
status: str
issues: List[Issue]

```

```

HARDWARE: Dict[str, Hardware] = {
    "h20-141": Hardware("h20-141", "NVIDIA H20 141 GB", "nvidia-vllm", 141),
    "h20-96": Hardware("h20-96", "NVIDIA H20 96 GB (beta)", "nvidia-vllm", 96),
    "910b-64": Hardware("910b-64", "Huawei Ascend 910B 64 GB", "vllm-ascend", 64),
}

```

```

MODELS: Dict[str, ModelProfile] = {
    "flash-0731": ModelProfile(
        key="flash-0731",
        label="DeepSeek V4 Flash 0731",
        context_limit=1_048_576,
        min_vllm_version="0.25.0",
        min_vllm_ascend_version="0.25.0",
        nvidia=ArtifactPlan(
            "deepseek-ai/DeepSeek-V4-Flash-0731",
            200.0,
            "official vLLM recipe vram_minimum_gb",
            4,
            "official 4-GPU recipe on H200/B200/B300; H20 is an extrapolation",
        ),
        ascend=ArtifactPlan(
            "DeepSeek-V4-Flash-0731-w8a8",
            None,
            "aggregate runtime floor not published; topology controls",
            8,
        ),
    ),
}

```

```

        "official vLLM-Ascend A2 topology: 1 node x 8 NPUs",
    ),
    v4_c4a_layers=21,
    v4_c128a_layers=20,
    v4_swa_layers=2,
),
"v4-pro": ModelProfile(
    key="v4-pro",
    label="DeepSeek V4 Pro",
    context_limit=1_048_576,
    min_vllm_version="0.20.0",
    min_vllm_ascend_version=None,
    nvidia=ArtifactPlan(
        "deepseek-ai/DeepSeek-V4-Pro",
        960.0,
        "official vLLM recipe vram_minimum_gb",
        8,
        "official 8-GPU H200 recipe; H20 is an extrapolation",
    ),
    ascend=ArtifactPlan(
        "DeepSeek-V4-Pro-w4a8-mtp",
        None,
        "aggregate runtime floor not published; topology controls",
        32,
        "official vLLM-Ascend A2 topology: 4 nodes x 8 NPUs",
    ),
    v4_c4a_layers=30,
    v4_c128a_layers=31,
),
"r1-671b": ModelProfile(
    key="r1-671b",
    label="DeepSeek R1 671B",
    context_limit=163_840,
    min_vllm_version="0.12.0",
    min_vllm_ascend_version=None,
    nvidia=ArtifactPlan(
        "deepseek-ai/DeepSeek-R1 (FP8)",
        805.0,
        "literal recipe vram_minimum_gb; its documented ceil example is
arithmetically inconsistent",
        8,
        "8-GPU FP8 serving topology; H20 performance is unverified",
    ),
    ascend=ArtifactPlan(
        "DeepSeek-R1-W8A8",
        None,
        "aggregate runtime floor not published; topology controls",
        16,

```

```

        "documented vLLM-Ascend A2 deployment: 2 x 8-NPU nodes, DP4/TP4/EP",
    ),
),
}

# These values are scenarios, not measurements. They are deliberately keyed by
# model and hardware so a real benchmark replaces exactly one deployment pair.
ILLUSTRATIVE_RATES: Dict[Tuple[str, str], ServiceRate] = {
    ("flash-0731", "h20-141"): ServiceRate(1_800, 180, "SIM"),
    ("flash-0731", "h20-96"): ServiceRate(1_600, 160, "SIM"),
    ("flash-0731", "910b-64"): ServiceRate(1_500, 150, "SIM"),
    ("v4-pro", "h20-141"): ServiceRate(1_200, 120, "SIM"),
    ("v4-pro", "h20-96"): ServiceRate(1_000, 100, "SIM"),
    ("v4-pro", "910b-64"): ServiceRate(900, 90, "SIM"),
    ("r1-671b", "h20-141"): ServiceRate(900, 90, "SIM"),
    ("r1-671b", "h20-96"): ServiceRate(750, 75, "SIM"),
    ("r1-671b", "910b-64"): ServiceRate(700, 70, "SIM"),
}

def parse_version(value: str, field: str) -> Version:
    try:
        return Version(value)
    except InvalidVersion as exc:
        raise ValueError(f"{field} must be a PEP 440 version, got {value!r}") from exc

def version_lt(left: str, right: str) -> bool:
    return parse_version(left, "version") < parse_version(right, "minimum version")

def validate_fraction(name: str, value: float) -> None:
    if not 0 < value <= 1:
        raise ValueError(f"{name} must be in (0, 1], got {value}")

def validate_workload(workload: Workload) -> None:
    if workload.users <= 0:
        raise ValueError("users must be positive")
    if workload.requests_per_user_hour <= 0:
        raise ValueError("requests_per_user_hour must be positive")
    if workload.peak_factor < 1:
        raise ValueError("peak_factor must be >= 1")
    if workload.input_tokens < 0 or workload.output_tokens <= 0:
        raise ValueError("input_tokens must be >= 0 and output_tokens must be positive")
    if workload.mean_residence_seconds <= 0:

```

```

        raise ValueError("mean_residence_seconds must be positive")

def artifact_for(
    model: ModelProfile, hardware: Hardware, speculative_mode: str = "none"
) -> ArtifactPlan:
    artifact = model.nvidia if hardware.runtime_family == "nvidia-vllm" else
model.ascend
    if (
        model.key == "v4-pro"
        and hardware.runtime_family == "nvidia-vllm"
        and speculative_mode == "dspark"
    ):
        return ArtifactPlan(
            "deepseek-ai/DeepSeek-V4-Pro-DSpark",
            960.0,
            "literal vLLM recipe vram_minimum_gb comparison for the DSpark variant",
            8,
            "official DSpark variant uses a distinct fused checkpoint; H20 remains
unverified",
            "0.25.0",
        )
    return artifact

def _resolved_v4_cache_layout(
    hardware: Hardware, kv_cache_dtype: str, indexer_cache_dtype: str
) -> Tuple[int, int, str, str]:
    resolved_kv = "fp8" if kv_cache_dtype == "auto" else kv_cache_dtype
    resolved_indexer = indexer_cache_dtype
    if resolved_indexer == "auto":
        resolved_indexer = "bf16" if hardware.runtime_family == "nvidia-vllm" else
"fp8"
    # vLLM's DeepSeek-V4 MLA spec keeps a semantic 512-element head for the
    # ordinary dtype path, so BF16 occupies 512 * 2 = 1,024 bytes. Only the
    # custom fp8_ds_mla layout stores the separate 448B NoPE + 128B RoPE + 8B
    # scale representation (584 bytes).
    main_entry = {"fp8": 584, "bf16": 1024}[resolved_kv]
    indexer_entry = round(128 * {"mxfp4": 0.5, "fp8": 1.0, "bf16":
2.0}[resolved_indexer])
    return main_entry, indexer_entry, resolved_kv, resolved_indexer

def kv_bytes_per_token(
    model: ModelProfile,
    hardware: Hardware,
    kv_cache_dtype: str,
    indexer_cache_dtype: str,

```

```

) -> Tuple[int, str]:
    if model.key == "r1-671b":
        resolved_kv = "bf16" if kv_cache_dtype == "auto" else kv_cache_dtype
        bytes_per_kv_element = {"fp8": 1.0, "bf16": 2.0}[resolved_kv]
        # Official R1 config: 61 layers x (kv_lora_rank 512 + rope dim 64).
        value = round(61 * (512 + 64) * bytes_per_kv_element)
        return value, f"{resolved_kv} MLA ({'auto from model dtype' if kv_cache_dtype
== 'auto' else 'explicit'})"

    main_entry, indexer_entry, resolved_kv, resolved_indexer =
_resolved_v4_cache_layout(
        hardware, kv_cache_dtype, indexer_cache_dtype
    )
    value = round(
        model.v4_c4a_layers * (main_entry + indexer_entry) / 4
        + model.v4_c128a_layers * main_entry / 128
    )
    return value, f"{resolved_kv} hybrid + {resolved_indexer} indexer (implementation-
informed estimate)"

def kv_fixed_bytes_per_sequence(
    model: ModelProfile,
    hardware: Hardware,
    kv_cache_dtype: str,
    indexer_cache_dtype: str,
) -> int:
    if model.key == "r1-671b":
        return 0
    main_entry, indexer_entry, _, _ = _resolved_v4_cache_layout(
        hardware, kv_cache_dtype, indexer_cache_dtype
    )
    # Each compressed layer retains 128 uncompressed shared-KV entries. C4
    # additionally retains an 8-entry indexer compressor residual. Flash has two
    # pure sliding-window layers. Runtime page-pool alignment can allocate more.
    return (
        model.v4_c4a_layers * (128 * main_entry + 8 * indexer_entry)
        + model.v4_c128a_layers * 128 * main_entry
        + model.v4_swa_layers * 128 * main_entry
    )

def padded_resident_tokens(workload: Workload, block_size: int) -> int:
    resident = workload.input_tokens + workload.output_tokens / 2
    return int(math.ceil(resident / block_size) * block_size)

def target_kv_gb(

```

```

bytes_per_token: int,
fixed_bytes_per_sequence: int,
workload: Workload,
block_size: int,
replicas: int,
) -> float:
    if replicas <= 0:
        raise ValueError("replicas must be positive")
    per_replica_concurrency = workload.concurrent_requests / replicas
    per_sequence = (
        padded_resident_tokens(workload, block_size) * bytes_per_token
        + fixed_bytes_per_sequence
    )
    return per_replica_concurrency * per_sequence / GB

def candidate_device_counts(hardware: Hardware, minimum: int, limit: int = 128) ->
Iterable[int]:
    local_groups = (1, 2, 4, 8)
    for count in local_groups:
        if count >= minimum:
            yield count
    first_multi_node = max(16, int(math.ceil(minimum / hardware.devices_per_node) *
hardware.devices_per_node))
    for count in range(first_multi_node, limit + 1, hardware.devices_per_node):
        yield count

def select_capacity_lower_bound_devices(
    artifact: ArtifactPlan,
    hardware: Hardware,
    utilization: float,
    workload_kv_gb: float,
) -> int:
    if workload_kv_gb < 0:
        raise ValueError("workload KV GB must be non-negative")
    # The recipe hint's margin composition is undefined, so adding KV to it
    # could double-count memory already covered by that margin. The maximum is
    # nevertheless a valid independent-constraint bound: the aggregate budget
    # must cover both the recipe hint and the modeled workload KV cache.
    required_gb = max(artifact.recipe_vram_hint_gb or 0.0, workload_kv_gb)
    for count in candidate_device_counts(hardware, artifact.documented_devices):
        if required_gb <= hardware.memory_gb * utilization * count:
            return count
    raise ValueError("aggregate capacity lower-bound search exceeded 128
accelerators")

```



```

def add_runtime_issues(
    issues: List[Issue],
    model: ModelProfile,
    hardware: Hardware,
    runtime: str,
    vllm_version: str,
    vllm_ascend_version: Optional[str],
    cann_version: Optional[str],
    artifact: ArtifactPlan,
) -> None:
    resolved = hardware.runtime_family if runtime == "auto" else runtime
    if resolved != hardware.runtime_family:
        issues.append(Issue("BLOCKER", "RUNTIME_MISMATCH", f"{resolved} cannot serve
{hardware.label}"))
        return
    if resolved == "nvidia-vllm":
        minimum = artifact.min_vllm_version_override or model.min_vllm_version
        if minimum and version_lt(vllm_version, minimum):
            issues.append(Issue(
                "BLOCKER",
                "VLLM_VERSION",
                f"vLLM {vllm_version} is below the official {minimum} minimum for this
artifact",
            ))
    else:
        if not vllm_ascend_version:
            issues.append(Issue("BLOCKER", "ASCEND_IMAGE_UNSPECIFIED", "supply a
matched vLLM-Ascend image/version"))
        elif model.min_vllm_ascend_version and version_lt(vllm_ascend_version,
model.min_vllm_ascend_version):
            issues.append(Issue(
                "BLOCKER",
                "VLLM_ASCEND_VERSION",
                f"vLLM-Ascend {vllm_ascend_version} is below
{model.min_vllm_ascend_version}",
            ))
        if not cann_version:
            issues.append(Issue("WARNING", "CANN_UNPINNED", "CANN/driver/firmware
versions are deployment parameters"))

def derive_status(issues: Sequence[Issue]) -> str:
    maximum = max((SEVERITY_RANK[item.severity] for item in issues), default=0)
    return {2: "BLOCKED", 1: "UNVERIFIED", 0: "ESTIMATE"}[maximum]

def size(
    model: ModelProfile,

```

```

hardware: Hardware,
workload: Workload,
*,
gpu_memory_utilization: float,
block_size: int,
max_num_seqs: int,
configured_devices: int,
runtime_reserve_gb_per_device: float,
kv_cache_dtype: str,
indexer_cache_dtype: str,
runtime: str,
vllm_version: str,
vllm_ascend_version: Optional[str],
cann_version: Optional[str],
reasoning_parser: str,
speculative_mode: str,
warm_standby: bool,
service_rate: Optional[ServiceRate],
) -> SizingResult:
    validate_fraction("gpu_memory_utilization", gpu_memory_utilization)
    validate_workload(workload)
    if block_size <= 0 or max_num_seqs <= 0 or configured_devices <= 0:
        raise ValueError("block_size, max_num_seqs, and configured_devices must be
positive")
    if runtime_reserve_gb_per_device < 0:
        raise ValueError("runtime_reserve_gb_per_device must be >= 0")
    if workload.input_tokens + workload.output_tokens > model.context_limit:
        raise ValueError(f"{model.label}: input + output exceeds the official context
limit")

    if service_rate and (service_rate.input_tps <= 0 or service_rate.output_tps <= 0):
        raise ValueError("service rates must be positive")
    concurrency_replicas = max(1, math.ceil(workload.concurrent_requests /
max_num_seqs))
    replicas: Optional[int] = None
    if service_rate:
        replicas = max(
            concurrency_replicas,
            math.ceil(workload.input_tps / service_rate.input_tps),
            math.ceil(workload.output_tps / service_rate.output_tps),
        )
    kv_replica_divisor = replicas or concurrency_replicas
    kv_bpt, kv_profile = kv_bytes_per_token(
        model, hardware, kv_cache_dtype, indexer_cache_dtype
    )
    kv_fixed = kv_fixed_bytes_per_sequence(
        model, hardware, kv_cache_dtype, indexer_cache_dtype
    )

```

```

kv_gb = target_kv_gb(
    kv_bpt, kv_fixed, workload, block_size, kv_replica_divisor
)
artifact = artifact_for(model, hardware, speculative_mode)
capacity_lower_bound = select_capacity_lower_bound_devices(
    artifact, hardware, gpu_memory_utilization, kv_gb
)

configured_usable = hardware.memory_gb * gpu_memory_utilization *
configured_devices
configured_required: Optional[float] = None
configured_headroom: Optional[float] = None
configured_fit: Optional[bool] = None
# The recipe hint's 20% margin has no defined composition. Adding KV or
# reserve to it (or treating it as an all-in floor) would invent semantics.
# These legacy fields therefore stay null until an exact runtime profile is
# supplied by a future topology-aware mode.

issues: List[Issue] = []
add_runtime_issues(
    issues,
    model,
    hardware,
    runtime,
    vllm_version,
    vllm_ascend_version,
    cann_version,
    artifact,
)
if hardware.runtime_family == "nvidia-vllm":
    issues.append(Issue(
        "WARNING",
        "H20_UNVERIFIED",
        "no cited official recipe validates this model on the supplied H20
variant",
    ))
    if model.key.startswith("flash") or model.key == "v4-pro":
        issues.append(Issue(
            "WARNING",
            "FP4_ON_H20",
            "native FP4 expert materialization and kernel behavior must be
measured on H20",
        ))
    else:
        issues.append(Issue(
            "NOTE",
            "DISTINCT_ARTIFACT",

```

```

        "Ascend topology uses a separately quantized artifact, not the NVIDIA
checkpoint",
    ))
    issues.append(Issue(
        "WARNING",
        "AGGREGATE_ONLY",
        "aggregate capacity is necessary but cannot establish per-rank TP/DP/EP/PP
residency or a runnable topology",
    ))
    if configured_devices < capacity_lower_bound:
        issues.append(Issue(
            "WARNING",
            "AGGREGATE_CAPACITY_SHORTFALL",
            f"configured {configured_devices} devices are below the aggregate bound
from the documented device minimum and max(recipe hint, workload KV); this still does
not establish per-rank fit",
        ))
    elif capacity_lower_bound < configured_devices:
        issues.append(Issue(
            "NOTE",
            "CAPACITY_BOUND_BELOW_CONFIGURED",
            f"aggregate capacity lower bound is {capacity_lower_bound}, but this does
not prove that {configured_devices} devices form a runnable topology",
        ))
    if artifact.recipe_vram_hint_gb is None:
        issues.append(Issue(
            "WARNING",
            "MEMORY_COMPOSITION_UNVERIFIED",
            "official topology is known, but checkpoint/activation/KV memory was not
independently profiled",
        ))
    if runtime_reserve_gb_per_device == 0:
        issues.append(Issue(
            "NOTE",
            "RUNTIME_RESERVE",
            "no extra per-device reserve was supplied; recipe hint margin composition
is undefined",
        ))
    if model.key != "r1-671b":
        issues.append(Issue(
            "NOTE",
            "HYBRID_KV_APPROXIMATION",
            "V4 KV includes documented compressed, sliding-window, and residual state
but not exact runtime page-pool padding",
        ))
    expected_parser = "deepseek_r1" if model.key == "r1-671b" else "deepseek_v4"
    if reasoning_parser != "auto" and reasoning_parser != expected_parser:

```

```

        issues.append(Issue("BLOCKER", "REASONING_PARSER", f"use {expected_parser} for
{model.label}"))
    if speculative_mode == "dspark" and not (
        model.key == "v4-pro" and hardware.runtime_family == "nvidia-vllm"
    ):
        issues.append(Issue(
            "BLOCKER",
            "SPECULATIVE_MODE",
            "DSpark is supported here only by the NVIDIA-vLLM Pro DSpark artifact",
        ))
    if speculative_mode != "none":
        issues.append(Issue(
            "WARNING",
            "SPECULATIVE_RATE",
            "service rate must be benchmarked with the selected draft method and
acceptance rate",
        ))
    if service_rate is None:
        issues.append(Issue(
            "WARNING",
            "SERVICE_RATE_REQUIRED",
            "fleet count is intentionally left TBD until a model+hardware service rate
is supplied",
        ))
    implied_decode_tps = workload.output_tokens / workload.mean_residence_seconds
    if implied_decode_tps > 200:
        issues.append(Issue(
            "WARNING",
            "RESIDENCE_TIME_INCONSISTENT",
            f"output/residence implies {implied_decode_tps:.0f} output tok/s per
request",
        ))

    return SizingResult(
        model_key=model.key,
        model=model.label,
        hardware_key=hardware.key,
        hardware=hardware.label,
        workload=workload.name,
        artifact=artifact.artifact,
        recipe_vram_hint_gb=artifact.recipe_vram_hint_gb,
        hint_basis=artifact.hint_basis,
        kv_profile=kv_profile,
        kv_bytes_per_token=kv_bpt,
        kv_fixed_bytes_per_sequence=kv_fixed,
        padded_resident_tokens=padded_resident_tokens(workload, block_size),
        fleet_concurrency=workload.concurrent_requests,
        concurrency_per_replica=workload.concurrent_requests / kv_replica_divisor,
    )

```

```

        kv_gb_per_replica=kv_gb,
        runtime_reserve_gb_per_device=runtime_reserve_gb_per_device,
        capacity_lower_bound_devices=capacity_lower_bound,
        capacity_lower_bound_nodes=math.ceil(capacity_lower_bound /
hardware.devices_per_node),
        capacity_lower_bound_usable_hbm_gb=capacity_lower_bound * hardware.memory_gb *
gpu_memory_utilization,
        configured_devices=configured_devices,
        configured_usable_hbm_gb=configured_usable,
        configured_required_hbm_gb=configured_required,
        configured_headroom_gb=configured_headroom,
        configured_memory_fit=configured_fit,
        required_fleet_input_tps=workload.input_tps,
        required_fleet_output_tps=workload.output_tps,
        required_input_tps_for_one_replica=(
            workload.input_tps / replicas if replicas is not None else None
        ),
        required_output_tps_for_one_replica=(
            workload.output_tps / replicas if replicas is not None else None
        ),
        service_input_tps=service_rate.input_tps if service_rate else None,
        service_output_tps=service_rate.output_tps if service_rate else None,
        service_rate_provenance=service_rate.provenance if service_rate else "TBD",
        throughput_replicas=replicas,
        candidate_fleet_devices=capacity_lower_bound * replicas if replicas else None,
        configured_fleet_devices=configured_devices * replicas if replicas else None,
        warm_standby_candidate_devices=capacity_lower_bound * replicas if warm_standby
and replicas else 0 if replicas else None,
        status=derive_status(issues),
        issues=issues,
    )

def parse_pair_key(value: str, field: str) -> Tuple[str, str]:
    try:
        model_key, hardware_key = value.split("@", 1)
    except ValueError as exc:
        raise ValueError(f"{field} must use MODEL@HARDWARE") from exc
    if model_key not in MODELS:
        raise ValueError(f"unknown model {model_key!r} in {field}")
    if hardware_key not in HARDWARE:
        raise ValueError(f"unknown hardware {hardware_key!r} in {field}")
    return model_key, hardware_key

def parse_service_rates(values: Sequence[str], illustrative: bool) -> Dict[Tuple[str,
str], ServiceRate]:
    rates = dict(ILLUSTRATIVE_RATES) if illustrative else {}

```

```

for value in values:
    try:
        pair, payload = value.split("=", 1)
        input_text, output_text, provenance = payload.split(",", 2)
        key = parse_pair_key(pair, "service-rate")
        provenance = provenance.upper()
        if provenance not in {"SIM", "MEAS"}:
            raise ValueError("provenance must be SIM or MEAS")
        rates[key] = ServiceRate(float(input_text), float(output_text),
provenance)
    except ValueError as exc:
        raise ValueError(
            f"invalid --service-rate {value!r}; use
MODEL@HARDWARE=INPUT,OUTPUT,SIM|MEAS"
        ) from exc
    return rates

def parse_replica_devices(values: Sequence[str], default_devices: int) ->
Dict[Tuple[str, str], int]:
    devices: Dict[Tuple[str, str], int] = {}
    for value in values:
        try:
            pair, count_text = value.split("=", 1)
            key = parse_pair_key(pair, "replica-devices")
            count = int(count_text)
            if count <= 0:
                raise ValueError("count must be positive")
            devices[key] = count
        except ValueError as exc:
            raise ValueError(
                f"invalid --replica-devices {value!r}; use MODEL@HARDWARE=COUNT"
            ) from exc
    for model_key in MODELS:
        for hardware_key in HARDWARE:
            devices.setdefault((model_key, hardware_key), default_devices)
    return devices

def workloads(args: argparse.Namespace) -> Iterable[Workload]:
    if len(args.users) != len(set(args.users)):
        raise ValueError("--users contains duplicate scenarios")
    for users in args.users:
        yield Workload(
            name=f"general-{users}-users",
            users=users,
            requests_per_user_hour=args.requests_per_user_hour,
            peak_factor=args.peak_factor,

```

```

        input_tokens=args.input_tokens,
        output_tokens=args.output_tokens,
        mean_residence_seconds=args.mean_residence_seconds,
    )

def run(args: argparse.Namespace) -> List[SizingResult]:
    rates = parse_service_rates(args.service_rate, args.illustrative_rates)
    configured = parse_replica_devices(args.replica_devices,
args.tensor_parallel_size)
    rows: List[SizingResult] = []
    for workload in workloads(args):
        for model_key in args.models:
            for hardware_key in args.hardware:
                rows.append(size(
                    MODELS[model_key],
                    HARDWARE[hardware_key],
                    workload,
                    gpu_memory_utilization=args.gpu_memory_utilization,
                    block_size=args.block_size,
                    max_num_seqs=args.max_num_seqs,
                    configured_devices=configured[(model_key, hardware_key)],
                    runtime_reserve_gb_per_device=args.runtime_reserve_gb_per_device,
                    kv_cache_dtype=args.kv_cache_dtype,
                    indexer_cache_dtype=args.indexer_cache_dtype,
                    runtime=args.runtime,
                    vllm_version=args.vllm_version,
                    vllm_ascend_version=args.vllm_ascend_version,
                    cann_version=args.cann_version,
                    reasoning_parser=args.reasoning_parser,
                    speculative_mode=args.speculative_mode,
                    warm_standby=args.warm_standby,
                    service_rate=rates.get((model_key, hardware_key)),
                ))
    return rows

def short_model(label: str) -> str:
    return label[len("DeepSeek "):] if label.startswith("DeepSeek ") else label

def short_hardware(label: str) -> str:
    for prefix in ("NVIDIA ", "Huawei Ascend "):
        if label.startswith(prefix):
            return label[len(prefix):]
    return label

```



```

def workload_groups(results: Iterable[SizingResult]) -> Iterable[Tuple[str,
List[SizingResult]]]:
    groups: Dict[str, List[SizingResult]] = {}
    for row in results:
        groups.setdefault(row.workload, []).append(row)
    return groups.items()

def fmt_optional(value: Optional[float], width: int = 8, decimals: int = 1) -> str:
    return f"{value:{width}.{decimals}f}" if value is not None else
f"{'n/a':>{width}}"

def print_memory_table(results: Iterable[SizingResult]) -> None:
    for index, (workload, rows) in enumerate(workload_groups(results)):
        if index:
            print()
        print(f"MEMORY/CAPACITY - {workload} - decimal GB")
        print(
            f"{'Model':15} {'Hardware':14} {'Hint':>7} {'KV/rep':>7} "
            f"{'Cap LB':>6} {'Cfg':>5} {'Status':>10}"
        )
        for row in rows:
            print(
                f"{short_model(row.model):15} {short_hardware(row.hardware):14} "
                f"{fmt_optional(row.recipe_vram_hint_gb, 7)} "
                f"{row.kv_gb_per_replica:7.2f} "
                f"{row.capacity_lower_bound_devices:6d} {row.configured_devices:5d} "
                f"{row.status:>10}"
            )
        print("\nHint = recipe authoring heuristic; Cap LB covers the documented minimum")
        print("and max(Hint, workload KV) without assuming those quantities are
additive.")
        print("Neither establishes per-rank fit or a runnable H20 topology.")

def print_capacity_table(results: Iterable[SizingResult]) -> None:
    for index, (workload, rows) in enumerate(workload_groups(results)):
        if index:
            print()
        print(f"CAPACITY - {workload}")
        print(
            f"{'Model':15} {'Hardware':14} {'Need in':>8} {'Need out':>8} "
            f"{'Rate':>5} {'Rep':>4} {'Cand fleet':>10} {'Cfg fleet':>9} "
            f"{'Status':>10}"
        )
        for row in rows:

```

```

        replicas = str(row.throughput_replicas) if row.throughput_replicas is not
None else "TBD"
        candidate = str(row.candidate_fleet_devices) if
row.candidate_fleet_devices is not None else "TBD"
        configured = str(row.configured_fleet_devices) if
row.configured_fleet_devices is not None else "TBD"
        print(
            f"{short_model(row.model):15} {short_hardware(row.hardware):14} "
            f"{row.required_fleet_input_tps:8.1f} "
            f"{row.required_fleet_output_tps:8.1f} "
            f"{row.service_rate_provenance:>5} {replicas:>4} {candidate:>10} "
            f"{configured:>9} {row.status:>10}"
        )
        print("\nTBD is intentional. Add --service-rate
MODEL@HARDWARE=INPUT,OUTPUT,MEAS.")
        print("--illustrative-rates enables explicit SIM scenarios; they are not
benchmarks.")

def print_issues(results: Iterable[SizingResult]) -> None:
    print("\nFINDINGS")
    seen: set[Tuple[str, str, str]] = set()
    for row in results:
        for issue in row.issues:
            key = (row.model_key, row.hardware_key, issue.code)
            if key in seen:
                continue
            seen.add(key)
            print(
                f"- [{issue.severity}] {short_model(row.model)} on "
                f"{short_hardware(row.hardware)} / {issue.code}: {issue.message}"
            )

def print_results(results: Iterable[SizingResult], view: str, show_issues: bool) ->
None:
    rows = list(results)
    if view in {"memory", "full"}:
        print_memory_table(rows)
    if view == "full":
        print()
    if view in {"capacity", "full"}:
        print_capacity_table(rows)
    if show_issues:
        print_issues(rows)

def self_test() -> None:

```

```

base = Workload("test", 200, 1, 3, 4096, 600, 30)
assert version_lt("0.25.0rc1", "0.25.0")
try:
    version_lt("latest", "0.25.0")
except ValueError:
    pass
else:
    raise AssertionError("unparseable versions must fail")
assert kv_bytes_per_token(MODELS["r1-671b"], HARDWARE["h20-141"], "fp8",
"auto")[0] == 35_136
assert kv_bytes_per_token(MODELS["r1-671b"], HARDWARE["h20-141"], "bf16",
"auto")[0] == 70_272
assert padded_resident_tokens(base, 256) == 4_608
one = target_kv_gb(35_136, 0, base, 256, 1)
two = target_kv_gb(35_136, 0, base, 256, 2)
assert math.isclose(one, two * 2)

common = dict(
    gpu_memory_utilization=0.75,
    block_size=256,
    max_num_seqs=256,
    configured_devices=8,
    runtime_reserve_gb_per_device=0,
    kv_cache_dtype="auto",
    indexer_cache_dtype="auto",
    runtime="auto",
    vllm_version="0.25.0",
    vllm_ascend_version="0.25.0",
    cann_version="9.0.1",
    reasoning_parser="auto",
    speculative_mode="none",
    warm_standby=False,
    service_rate=ServiceRate(1_000, 100, "MEAS"),
)
flash = size(MODELS["flash-0731"], HARDWARE["h20-141"], base, **common)
pro = size(MODELS["v4-pro"], HARDWARE["h20-141"], base, **common)
r1_96 = size(MODELS["r1-671b"], HARDWARE["h20-96"], base, **common)
assert flash.capacity_lower_bound_devices == 4 and flash.configured_devices == 8
assert pro.capacity_lower_bound_devices == 16 and pro.configured_memory_fit is
None
assert r1_96.capacity_lower_bound_devices == 16
assert kv_bytes_per_token(MODELS["r1-671b"], HARDWARE["h20-141"], "auto",
"auto")[0] == 70_272
assert pro.throughput_replicas == 1
assert any(issue.code == "AGGREGATE_CAPACITY_SHORTFALL" for issue in pro.issues)
try:
    validate_workload(Workload("bad", -1, 1, 1, 1, 1, 1))
except ValueError:

```

```

        pass
    else:
        raise AssertionError("negative users must fail")
    parsed = parse_service_rates(["flash-0731@h20-141=10,20,MEAS"], False)
    assert parsed[("flash-0731", "h20-141")].provenance == "MEAS"
    print("self-test: PASS")

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="DeepSeek memory, topology, and conditional fleet sizing.",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
    )
    parser.add_argument("--models", nargs="+", choices=list(MODELS),
default=list(MODELS))
    parser.add_argument("--hardware", nargs="+", choices=list(HARDWARE),
default=list(HARDWARE))
    parser.add_argument("--users", type=int, nargs="+", default=[200])
    parser.add_argument("--requests-per-user-hour", type=float, default=1.0)
    parser.add_argument("--peak-factor", type=float, default=3.0)
    parser.add_argument("--input-tokens", type=int, default=4096)
    parser.add_argument("--output-tokens", type=int, default=600)
    parser.add_argument("--mean-residence-seconds", type=float, default=30.0)
    parser.add_argument(
        "--gpu-memory-utilization",
        type=float,
        default=0.75,
        help="supplied KPMG policy; upstream vLLM currently defaults to 0.92",
    )
    parser.add_argument("--block-size", type=int, default=256)
    parser.add_argument("--max-num-seqs", type=int, default=256)
    parser.add_argument("--tensor-parallel-size", type=int, default=8)
    parser.add_argument(
        "--replica-devices",
        action="append",
        default=[],
        metavar="MODEL@HARDWARE=COUNT",
        help="override the configured device count for one deployment pair",
    )
    parser.add_argument("--runtime-reserve-gb-per-device", type=float, default=0.0)
    parser.add_argument(
        "--kv-cache-dtype",
        choices=["auto", "fp8", "bf16"],
        default="auto",
        help="auto follows the selected recipe/model: V4 FP8, R1 model BF16",
    )
    parser.add_argument(
        "--indexer-cache-dtype",

```

```

        choices=["auto", "mxfp4", "fp8", "bf16"],
        default="auto",
    )
    parser.add_argument("--runtime", choices=["auto", "nvidia-vllm", "vllm-ascend"],
default="auto")
    parser.add_argument("--vllm-version", default="0.20.1")
    parser.add_argument("--vllm-ascend-version")
    parser.add_argument("--cann-version")
    parser.add_argument(
        "--reasoning-parser",
        choices=["auto", "deepseek_v4", "deepseek_r1"],
        default="auto",
    )
    parser.add_argument("--speculative-mode", choices=["none", "mtp", "dspark"],
default="none")
    parser.add_argument(
        "--service-rate",
        action="append",
        default=[],
        metavar="MODEL@HARDWARE=INPUT,OUTPUT,MEAS",
        help="repeatable model+hardware service rate; provenance is SIM or MEAS",
    )
    parser.add_argument(
        "--illustrative-rates",
        action="store_true",
        help="use built-in pair-specific SIM rates; not suitable for procurement",
    )
    parser.add_argument("--warm-standby", action="store_true")
    parser.add_argument("--view", choices=["memory", "capacity", "full"],
default="memory")
    parser.add_argument("--show-issues", action="store_true")
    parser.add_argument("--show-blockers", action="store_true", help="compatibility
alias for --show-issues")
    parser.add_argument("--json", action="store_true")
    parser.add_argument("--self-test", action="store_true")
    return parser.parse_args()

def main() -> None:
    args = parse_args()
    if args.self_test:
        self_test()
        return
    try:
        results = run(args)
    except ValueError as exc:
        raise SystemExit(f"error: {exc}") from exc
    if args.json:

```

```
        print(json.dumps([asdict(result) for result in results], indent=2))
    else:
        print_results(results, args.view, args.show_issues or args.show_blockers)

if __name__ == "__main__":
    main()
```